Python Engineering - Engineering & Math Theory

Class 11 - Coordinate Transforms: 2D rotation theory

This document summarizes the mathematics and physics behind EX19. The goal is to understand what a rotation represents and why the matrix has its main properties, then recognize the same ideas in the NumPy implementation.

1. What problem are we solving?

We have a vector v in a Cartesian plane and want to rotate it by an angle theta while keeping the axes fixed. EX19 uses an **active rotation**: the vector changes, not the reference frame.

If the original vector is v = [x, y], we seek v' = [x', y'] with the same magnitude but a direction rotated by theta.

2. Where do the rotation equations come from?

Write the vector using magnitude r and angle phi:

$$x = r\cos(\phi) \qquad y = r\sin(\phi)$$

After rotating by theta, the new angle is phi + theta:

$$x' = r\cos(\phi + \theta)$$
$$y' = r\sin(\phi + \theta)$$

Using the angle-sum trigonometric identities:

$$\cos(\phi + \theta) = \cos(\phi)\cos(\theta) - \sin(\phi)\sin(\theta)$$
$$\sin(\phi + \theta) = \sin(\phi)\cos(\theta) + \cos(\phi)\sin(\theta)$$

Substituting x = r cos(phi) and y = r sin(phi):

$$x' = x\cos(\theta) - y\sin(\theta)$$
$$y' = x\sin(\theta) + y\cos(\theta)$$

3. Matrix form

The two equations can be written as one matrix multiplication:

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

$$v' = R(\theta)\, v = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} [x, y]^T$$

This is exactly what we implement with `R @ vector`.

4. Manual example: rotate [100,0] by 90 deg

For theta=90 deg, cos(90)=0 and sin(90)=1:

$$R(90) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$x' = 0*100 - 1*0 = 0$$
$$y' = 1*100 + 0*0 = 100$$

Result: [100,0] -> [0,100], matching reference case A.

5. Why does a rotation preserve magnitude?

Physically, rotating a vector should not make it longer or shorter. Its Euclidean norm is:

$$||v||^2 = x^2 + y^2$$

After rotation:

$$||v'||^2 = (x \cos(\theta) - y \sin(\theta))^2 + (x \sin(\theta) + y \cos(\theta))^2$$

Expanding the terms cancels the cross terms and uses cos^2(theta)+sin^2(theta)=1, giving:

$$||v'||^2 = x^2 + y^2 = ||v||^2$$

That is why EX19 compares the vector norm before and after the rotation using `np.linalg.norm()` and `np.allclose()`.

6. Orthogonality of a rotation matrix

A matrix is orthogonal when its columns are unit vectors perpendicular to each other. For R(theta), the columns are:

$$c1 = [\cos(\theta), \sin(\theta)]$$
$$c2 = [-\sin(\theta), \cos(\theta)]$$

Each has norm 1 because cos^2+sin^2=1, and their dot product is zero:

$$c1 \cdot c2 = -\cos(\theta)\sin(\theta) + \sin(\theta)\cos(\theta) = 0$$

Therefore the columns form an orthonormal basis and:

$$R^T R = I$$

7. Simple proof that R^T = R^-1

The inverse matrix R^-1 is defined by:

$$R^{-1} R = I$$

But an orthogonal rotation matrix also satisfies:

$$R^T R = I$$

Therefore:

$$R^{-1} = R^T$$

Physical interpretation: if R rotates by +theta, R.T undoes that rotation. In EX19 we compute `recovered = R.T @ rotated`.

8. Determinant of a rotation

For a 2x2 matrix det([[a,b],[c,d]]) = ad-bc. For R(theta):

$$\det(R) = \cos^2(\theta) + \sin^2(\theta) = 1$$

A determinant of +1 identifies a proper rotation: lengths, angles, and orientation are preserved. An orthogonal reflection would have determinant -1.

9. Consecutive rotations

Rotate first by theta1 and then by theta2:

$$v'' = R(\theta_2)\, R(\theta_1)\, v$$

In 2D:

$$R(\text{theta2})\, R(\text{theta1}) = R(\text{theta1} + \text{theta2})$$

This is why EX19 checks that 20 deg followed by 40 deg equals 60 deg.

10. Manual example: rotate [3,4] by 30 deg

For v=[3,4], theta=30 deg, cos(30)=0.866025 and sin(30)=0.5:

$$x' = 3(0.866025) - 4(0.5) = 0.5981$$
$$y' = 3(0.5) + 4(0.866025) = 4.9641$$

The original norm is sqrt(3^2+4^2)=5, and the rotated vector also has norm approximately 5.

11. Numerical error and theoretical zeros

Mathematically cos(90 deg)=0 exactly. A computer may produce 6.123e-17 because real numbers are represented with finite precision. This does not mean the rotation is wrong.

That is why tolerance-based checks such as `np.allclose()` are useful.

12. How does this theory connect to EX19?

EX19 block	Theory used
A	Degrees -> radians for numerical trigonometry
B	Construction of R(theta)
C/D	Matrix product v' = Rv
E	Norm preservation
F	Orthogonality and inverse R^-1 = R^T
G	Composition R(theta2)R(theta1)=R(theta1+theta2)
3D bonus	Extension of the same idea to rotation about a 3D axis

Final idea

A rotation matrix is not merely a table of sines and cosines: it is a linear transformation that preserves the geometry of a vector. The properties checked in Python - magnitude, inverse, and composition - follow directly from that mathematical structure.